UX RECOVERY PROGRAM · FORENSIC AUDIT + RESKIN SPECIFICATION

SupportOS-Plus UX Recovery & Reskin Plan

Why the interface reads 0/10 - traced to file:line receipts - and the Intercom-grade presentation plan that makes the program fathomable again without changing a single feature, table, or sync behavior.

Independent UX Audit & Design Specification

Prepared for the SupportOS-Plus project · kimpearce888/SupportOS-Plus

OCTOBER 2026 · COMMIT EE6B35A · V1.0

Table of Contents

1. Executive Summary 3

2. Forensic Audit: Why the Inbox Reads 0/10 3

2.1 The thread that cannot be read . . . 4

2.2 AI responses that look like human replies . . . 4

2.3 The data path under the mess . . . 4

2.4 Triage affordances that do not exist . . . 5

2.5 Navigation, search and deep links . . . 6

3. The Same Failure Program-Wide 6

4. What Is Already Good (Keep List) 7

5. Root Causes 7

6. The Reference Standard: the Intercom Model 8

7. Scope Contract 9

8. Reskin Plan R0-R5 9

9. R0 Reference Specification 10

9.1 Design tokens v3 - semantic roles . . . 10

9.2 Message card anatomy (the core component) . . . 11

9.3 AI answer card . . . 11

9.4 Conversation list row . . . 11

9.5 Composer . . . 12

9.6 Copilot panel . . . 12

9.7 QA gates . . . 12

10. The Interaction Track (Fast-Follow) 12

11. Metrics, Effort and Sequencing 13

12. Do-NOT List 14

A. Appendix A. Receipt Index and Source Register 14

A.1 Receipt index 14

A.2 Source register 15

1. Executive Summary

This report answers the owner's verdict that SupportOS-Plus 'is not at all user-friendly - it is a mess, specially the way it handled the inbox - 0/10', and his refinement of the goal: 'the program itself is not in the question; how the design is presenting the info all over the program is not fathomable; keeping everything the same but reskinning it with something industry standard like Intercom is the goal.' The report therefore does two things. First, it treats the 0/10 as a measurable engineering claim and proves it from the interface source itself, with a file:line receipt for every defect (Chapters 2-3). Second, it delivers a complete reskin plan - phases R0-R5 with an embedded R0 design specification - that rebuilds the presentation layer to the semantic standard set by Intercom's Inbox and the wider agent-workspace category, while changing no feature, no database table, no sync behavior, and no workflow (Chapters 7-9).

The single most important technical finding is structural: **the presentation layer has no 'message' object.** Every entry in a conversation - customer email, agent reply, internal note, AI answer - is rendered by one flat function that paints an 11-pixel badge, a name, a raw UTC timestamp, then a wall of full-width text, then a 1-pixel divider (inbox.rs:2858-2930). Because every block is typed only by that tiny badge - and AI entries share the exact same color tone as agent replies (inbox.rs:2871 vs 2875) - a reader literally cannot see where one response ends and the next begins, nor which voice is speaking. The data needed to fix this already exists in every row (ThreadEntry carries type, author, role and timestamp - core/inbox.rs:326-341); the interface simply discards it. That is why the right recovery is a reskin plus one new component class - the role-typed message card - not a rewrite, and not a mere color swap.

1 function
renders every thread entry - no message object exists (inbox.rs:2858)

6+ queries
run on the UI thread per conversation click (inbox.rs:526-676)

1 tone
shared by Agent and AI badges - AI is invisible (inbox.rs:2871/2875)

0 changes
to features, DB, sync or flows in the reskin scope

Everything in this document was re-derived on 28 October 2026 from the repository at commit ee6b35a (main, kimpearce888/SupportOS-Plus) and from live research: Intercom's own Inbox and Fin documentation, Zendesk Agent Workspace material, chat-UI and AI-agent pattern libraries, the egui show_rows / egui_kittest ecosystem, and usability-measurement literature (SUS benchmarks, Nielsen Norman heuristics). Chapter 12's do-NOT list, Chapter 11's metrics and the R0 specification are written so that a designer and a developer can start the following morning without another planning round.

2. Forensic Audit: Why the Inbox Reads 0/10

This chapter walks the actual rendering code in order of what a user hits first: the list, then the thread, then the composer. Every claim carries a file:line receipt from the audit clone (crates/app-shell unless noted). The point of the receipts is not blame - it is to make the recovery plan falsifiable: each defect below maps to a line in the R0-R5 plan that removes it.

2.1 The thread that cannot be read

The function thread_entry_ui draws every conversation entry the same way (inbox.rs:2858-2930): a horizontal line with an 11 px badge ('Customer' / 'Agent' / 'Note' / 'AI'), the actor name at 13 px, and a right-aligned raw UTC timestamp; below it, the HTML body is flattened by richtext::html_to_spans into plain styled spans painted edge-to-edge with no container; below that, a 1 px divider (inbox.rs:2929). There is no bubble, no card, no tinted background, no avatar, no left accent, no date separator, no grouping of consecutive messages, and no padding between the author line and the body. Distinguishing who is speaking relies entirely on reading an 11-pixel label. Quoted e-mail history - the very thing that makes real support threads unreadable everywhere else too - is rendered inline as weak-colored text (inbox.rs:2916-2918), so old messages, signatures and quote-backs blur straight into new responses. This is the mechanical explanation for 'you will not have any idea where first response ended where the next response started'.

Compounding the flatness, the entire thread is read through a scroll card capped at 300 px inside an already-scrolling pane (inbox.rs:2306-2314, .max_height(300.0)), while the loader fetches every entry's full HTML body with no LIMIT (core/inbox.rs:326-341, ORDER BY created_at ASC). A 100-message conversation costs a hundred full bodies in memory to show a fourteen-row keyhole. Timestamps are raw ISO strings truncated to 'YYYY-MM-DD HH:MM' in UTC (compact_time, inbox.rs:2797-2801) - no relative time, no timezone conversion, no day grouping.

2.2 AI responses that look like human replies

The audit-fix comment at inbox.rs:2864-2878 correctly enumerates real Help Scout thread types (customer, message, note, chat, phone, lineitem) and maps anything starting with 'ai' to an 'AI' badge. But the tone chosen for AI is BadgeTone::Info - the identical tone used for Agent (inbox.rs:2871 vs 2875). In both light and dark themes the AI badge is the same soft blue pill as the agent badge, in the same position, at the same size, next to the same kind of text. An AI answer and a human answer are visually indistinguishable unless the user reads every badge. The Copilot side panel (inbox.rs:1473-1589) compounds the problem: it is one dense card in which signal chips, a bullet summary, open questions, history hints, similar-issue rows and a nested-scroll suggested reply all render at the same small size with the same visual weight - there is no hierarchy between 'what the machine noticed' and 'what the machine proposes you send'. Composer AI actions (rewrite, translate, coaching - inbox.rs:2442-2500, guard rule at :398) silently replace composer text with no persistent 'AI edited this' state.

2.3 The data path under the mess

Every inbox interaction runs SQLite synchronously on the UI thread. Opening the page issues the list query inside db.with_read on the render path (inbox.rs:526-529); selecting a row triggers at least five more synchronous queries (detail at :551, tags at :574, and further with_read blocks at :624 and :676), including a copilot report that runs vector plus FTS search on-thread when the panel is open (:598-605). Two hot loops make the page actively hostile in its edge states: if the list is empty - a legitimate zero-hit filter or a fresh pre-sync database - the reload runs again on every rendered frame (inbox.rs:1051-1056, roughly sixty COUNT+SELECT pairs per second of mouse movement, sustained by the 1.5 s repaint heartbeat); and if the

selected conversation failed to load or was deleted, the six-query detail reload repeats every frame forever (:1060-1062). The list itself is a hand-painted for-loop over all 50 rows with a per-frame clone of the whole row vector (inbox.rs:1288-1290, allocate_space 66.0 px rows at :1293), not egui's built-in virtualized show_rows.

Defect	What the user experiences	Receipt
Synchronous UI-thread SQL on every click	Freeze on every filter / pager / row click	inbox.rs:526-529, 551, 574, 624, 676; db.rs:184
Empty-state hot loop	Battery drain and stutter whenever a filter matches zero rows	inbox.rs:1051-1056
Broken-detail hot loop	Permanent re-query when a conversation errors or vanished	inbox.rs:1060-1062
Non-virtualized hand-painted list + per-frame clone	Wasted work per frame; scaling ceiling	inbox.rs:1288-1290, 1293
Thread: unlimited load, 300 px window	Long conversations unreadable and expensive at once	inbox.rs:2306-2314; core/inbox.rs:326-341
Fixed sort on unindexed COALESCE	Full sort per reload; no user sort control	core/inbox.rs:234; migrations.rs:100-105
LIKE-scan inbox search	Weakest search in the app while an FTS5 engine exists	core/inbox.rs:197-201; search.rs

Table 2-1. Data-path and interaction defects with receipts.

2.4 Triage affordances that do not exist

An inbox is a triage instrument, and this one ships without the instruments. There is no unread concept anywhere (the schema has no read flag; ConversationListItem carries none - core/inbox.rs:59-81). Rows paint only subject, status badge, customer and mailbox (inbox.rs:1316-1374) even though the row struct already carries assignee and priority fields (core/inbox.rs:68-76); the assignee filter is hardcoded to None in the UI (inbox.rs:121) and priority is never exposed, so 'my conversations' or 'assigned to teammate X' cannot be expressed. There is no multi-select and no bulk action of any kind; no saved views inside the inbox; no sort control (ORDER BY is fixed); pagination is 50 per page with prev/next only (INBOX_PAGE_SIZE at inbox.rs:92, pager at :1427-1435), so reaching conversation #1,900 takes 38 clicks. Saved Replies - the number-one accelerator in helpdesk work - exist as CRUD on the Operations page (operations.rs:746-747) but are unreachable from the composer: a search for 'saved' in inbox.rs returns zero hits. Drafts live only in memory and 'the stash never outlives the session' (inbox.rs:373-390): a crash or quit mid-reply silently destroys the customer's answer, in the exact scenario the app's own fatal.rs crash handler exists for. Attachments are auto-downloaded by a background worker (workers.rs:3857-3918) but never shown in the reader.

2.5 Navigation, search and deep links

The command palette parses /inbox/conversation/{id} and then discards the id - palette.rs:92-96 maps every parse to a generic NavRequest::Page(Inbox); customers and knowledge hits lose their ids the same way (:98-106). Searching 'invoice #778' and pressing Enter lands on an unfiltered Inbox - the promise 'jump to the thing' is broken at the last hop. The sidebar groups 23 pages with reasonable intent (app.rs:926-971) but its System group is a nine-item dumping ground; filter state lives only in RAM (FilterBar never persisted, inbox.rs:73-87); there are no bookmarks, recents, or window-title sync. Keyboard support exists and is good where present - Ctrl/Cmd+K palette (palette.rs:185-187), Ctrl+Enter send (inbox.rs:2406-2410), mention-popup arrows (2370-2382) - but there is no j/k list navigation, no next-conversation key, no shortcut cheatsheet anywhere.

3. The Same Failure Program-Wide

The inbox is the worst case, not the exception. The list rows across the program are hand-painted three-line blocks with magic numbers re-declared per call site - 66.0 px rows in the inbox (inbox.rs:1293), a 360 px list panel (:1266), a 216 px sidebar (app.rs:721), a 52 px header (:738), 22/14 px page margins (:782) - instead of one spacing scale. Pages are uniform stacks of identically-styled cards: nothing tells the eye what is primary, what is secondary, and what is machinery. Empty states exist (design::empty_state) but are inconsistently applied; loading indicators appear on some pages while others simply freeze; error text surfaces in some flows while sync errors only reach toasts and the Diagnostics page. Roughly 460 English UI strings are hardcoded (design::button 102, hint_text 38, on_hover_text 29, RichText 21 plus page literals), the font set ships without CJK coverage (design.rs:210-218), and every timestamp in the program is un-localized UTC. The organization the interface needs - a visual grammar of primary/secondary/tertiary information - has never been specified.

One honest boundary belongs here: this report's predecessor already proved that the styling foundation is not the villain. The design.rs token system (zinc LIGHT/DARK tables, badge, chip, card, pager, page_header, empty_state components - design.rs:1-10, 104-179, 547-911), the toast system with per-kind lifetimes, the typed destructive-action confirm dialog covering 12 actions, the duplicate-send guard and the honest outbound status line are genuinely good. Only six stray hardcoded colors exist in the whole shell (settings.rs:1820-1826). The 0/10 is therefore a failure of information architecture and interaction presentation wearing a decent coat - which is precisely why an Intercom-style reskin that adds the missing semantic layer can work without touching the program.

4. What Is Already Good (Keep List)

A recovery plan that does not name its assets will destroy them. The following are explicitly preserved by the reskin; several become the substrate the new components are built from.

Asset	Why it is kept as-is	Receipt
design.rs token system + component family	Centralized zinc LIGHT/DARK tables; badge/chip/card/pager/empty_state; the reskin extends, not replaces	design.rs:1-10, 104-179, 547-911
Theme persistence + instant switch	SQLite-backed theme row; both themes fully tokenized	theme.rs:23, 59-63; design.rs:238-239
Toast system	Per-kind auto-dismiss 4/6/10 s with hidden-window lifetime fix; loud failure feedback	toasts.rs:39-46, 154; app.rs:684-692
Destructive-action confirm dialog	Typed 12-action enum gates every destructive flow	pages/mod.rs:47-75
Duplicate-send protection + honest send status	sha-key guard; 'Reply delivered / failed' line - exactly the trust pattern Intercom-style products use	inbox.rs:2954-2971, 2288-2294
Mention autocomplete + AI chips	Keyboard-complete @mentions; rewrite/translate present (presentation only needs fixing)	inbox.rs:2370-2382, 2442-2500
Command palette	Ctrl/Cmd+K, arrows, Esc; deep-link wiring is the fix, not the concept	palette.rs:185-187, 388-391
a11y gestures	WidgetInfo roles on nav rows and list items - to be completed, not started	app.rs:1046-1048; inbox.rs:1409-1418
Screenshot harness + smoke gate	SPP_SHOT_* knobs and 29 PNG baselines; becomes the reskin's regression instrument	app.rs:1180-1189; qa-redesign/screenshots/

Table 4-1. Assets preserved by the reskin.

5. Root Causes

Four root causes produced the mess, and each one dictates a property of the plan.

- **The UI mirrors the database, not the job.** Pages correspond to tables; the thread renders whatever the row struct happens to contain in whatever order the painter was written. Nothing asked 'what does an agent need to see in the first 200 ms?' The reskin therefore starts from role-typed message objects, not from colors.
- **No measurement loop.** No SUS baseline, no task tests, no heuristic evaluation was ever run - '0/10' was unfalsifiable until now. The plan pins numbers first (Chapter 11) so recovery is provable, and re-tests on the same instrument.
- **Immediate-mode primitives without widget discipline.** egui gives painter-level freedom; without a component library it invites hand-painted rows and per-call-site magic numbers. The plan adds exactly

one new component class (message card) and extends design.rs so the freedom is never used raw again.

- **Performance debt experienced as UX debt.** Synchronous UI-thread SQL does not feel like an architecture issue to a user; it feels like 'this program is a mess'. The reskin is scoped to presentation, and the async data path is explicitly scheduled as the fast-follow interaction track (Chapter 10) - both ships, in that order.

6. The Reference Standard: the Intercom Model

'Industry standard like Intercom' resolves, on inspection of Intercom's own product documentation and design material (all fetched 28 October 2026, register in Appendix A), into five concrete, transferable conventions. None of them require copying Intercom's palette; the transferable asset is the semantic layer.

- **A shared, omnichannel workspace.** Intercom's Inbox is defined as 'the workspace where your team manages and responds to customer conversations', organized by views (curated filters such as 'Escalated & Handoff'). The unit of work is the conversation; the list is a triage surface, not a table dump.
- **Typed message blocks, not chat bubbles.** In agent workspaces (Intercom Inbox, Zendesk Agent Workspace) thread entries render as full-width blocks whose authorship is unmistakable: avatar + name + role for customers and teammates, and **internal notes as yellow-tinted private blocks on the same thread**. Intercom's composer even defaults to Note when the last entry was a note - state follows the thread.
- **AI output is its own card type.** Fin answers carry a distinct AI identity, provenance and explicit follow-up actions; Intercom's design leadership treats 'Fin-ness' as a deliberate visual brand. AI-agent UI pattern libraries agree: suggested replies render as editable cards with Insert / Regenerate / Copy actions and a visible provenance line - never as lookalike human text.
- **Keyboard-first operation.** The category benchmark (Superhuman's mail model: command bar, 100+ shortcuts, hover-to-reveal shortcut teaching) sets user expectations; a support inbox competes with that muscle memory whether it wants to or not.
- **Measured usability.** The category instrument is the System Usability Scale: 68 is the 50th percentile, above 80.3 is the top decile, below 50 is conventionally unacceptable (Brooke; 2025 psychometric review). A product that feels 0/10 typically scores in the 30s - which makes 'reskin to at least 68' a statement about testable outcomes, not taste.

Two egui-native facts make this standard achievable without leaving the toolkit: ScrollArea::show_rows is a built-in virtualized list (the official documentation demonstrates it with ten thousand rows), and egui_kittest - the snapshot-testing library based on AccessKit that egui has shipped since 0.33 - provides the missing UX regression gate. The existing eframe 0.37 dependency already compiles AccessKit in by default.

7. Scope Contract

The owner's instruction - keep everything the same, reskin the presentation - is formalized as the following contract. It is the plan's definition of done and its change-control fence.

Dimension	Status in reskin	Notes
Features, pages (23), workflows	UNCHANGED	No feature added, removed, or relocated
Database schema, sync engine, queries	UNCHANGED	No migration ships in this workstream
Keyboard shortcuts, composer behavior	UNCHANGED	Ctrl+Enter send, mention autocomplete, AI chips all keep semantics
design.rs tokens + component family	EXTENDED	Token v3 adds semantic roles; existing names keep meaning
Thread renderer	REBUILT (presentation only)	Same ThreadEntry struct, same queries; new message-card presentation
List rows, toolbar, pager, filters	RESTYLED	Existing fields only (status, assignee, mailbox, times)
Copilot panel, AI chips	RESTYLED into typed cards	Same data, provenance line added
Unread flags, bulk actions, async data path, i18n, IA changes	OUT OF SCOPE	Deferred to the interaction track (Chapter 10)

Table 7-1. The scope contract.

8. Reskin Plan R0-R5

Six phases, roughly eight weeks elapsed part-time, one product designer plus one Rust developer. Every phase ends with an artifact and an exit test; nothing advances on taste.

Phase	Week	Deliverables	Exit criteria
R0 Reference spec	1	Written spec distilled from Intercom Inbox / Zendesk AW anatomy, wireframed against the app's real fields (mock 20-message thread with notes and AI entries)	A stranger segments the mock thread correctly in 5 seconds; owner signs the spec
R1 Token v3	2	design.rs extensions: semantic roles (Table 9-1), radius scale, 4-pt spacing, type scale; LIGHT + DARK mirrored	Token diff table reviewed; zero call-site edits needed for base reskin
R2 Message card + thread rebuild	3-4	design::message_card component; thread pane becomes full-height; quoted-text collapse; date separators; attachments strip	Blind thread-segmentation test passes; kittest snapshots for the card land
R3 Inbox chrome	5	List rows, segmented view control, restyled filters/pager, Reply|Note composer tabs, AI-edited chip, Copilot panel split into typed cards	All six golden tasks unchanged in behavior; screenshots regenerated

Phase	Week	Deliverables	Exit criteria
R4 Program-wide reskin	6-7	Remaining 22 pages restyled via tokens: headers, cards, tables, forms, empty states, toasts, palette, sidebar	Color-lint: zero colors outside tokens; screenshot set regenerated
R5 Gates & validation	8	egui_kittest snapshots in CI; golden screenshots via existing ui-screenshot harness; a11y labels; SUS re-test	Chapter 11 metrics met; regression count 0

Table 8-1. Phase plan with exits.

9. R0 Reference Specification

This chapter is the design contract the R0 phase formalizes. Values are proposals in zinc terms consistent with the existing token tables; the designer's first deliverable is to confirm them against real content and hand them to R1.

9.1 Design tokens v3 - semantic roles

Token	Light	Dark	Applied to
surface	#ffffff	#0f1418	Window, panes
surface.raised	#f4f4f5	#181d23	Cards, message bodies
bubble.customer	#f4f4f5	#1c222a	Customer message card fill
bubble.agent	#e8f2fa	#16283a	Agent reply card fill (primary-tinted)
bubble.note	#fef9c3	#3a3313	Internal note card fill (Intercom yellow convention)
bubble.ai	#f3effb	#251c3a	AI answer card fill (distinct violet accent)
bubble.system	transparent	transparent	System/meta entries, mono text
divider.strong	#d4d4d8	#2a323c	Between conversation groups
accent.ai	#7c5cbf	#a78bda	AI badge, AI accent edge, provenance text

Table 9-1. Semantic role tokens (zinc-based proposals).

Rules: radius scale 6 (chips) / 8 (inputs, small cards) / 12 (message cards) / 16 (panes); spacing on a 4-pt grid with named steps 4, 8, 12, 16, 24, 32; type scale 11 / 12 / 13 / 14 / 16 / 20 px with 13 px as body and 11 px reserved for meta. Every magic number currently re-declared per call site (66.0 rows, 360 px panel, 300 px cap, 216 px sidebar, 52 px header) maps to a named token and loses its literal.

9.2 Message card anatomy (the core component)

One card per thread entry, replacing the flat badge-line-wall-divider rendering of thread_entry_ui (inbox.rs:2858-2930). Zones, top to bottom:

- **Header row:** 28 px avatar circle with initials (customer) or role glyph (agent, note, AI) + author display name (13 px semibold) + role chip (Customer / Agent / Note / AI / System) + relative local timestamp ('3m ago', 'Yesterday 14:02') right-aligned; full UTC on hover.
- **Body:** rendered inside the role-tinted card (radius 12, 12-16 px padding); rich spans keep bold/italic/code/links behavior exactly as today.
- **Quoted region:** detected quote-backs collapse behind a 'Show quoted lines' toggle with a count - quotes can never again masquerade as a new response (fixes inbox.rs:2916-2918 behavior).
- **Attachment strip:** chips for every downloaded attachment of the entry (files already on disk via workers.rs:3857-3918) with open-in-file-manager action.
- **Grouping and rhythm:** consecutive entries by the same author merge into one card with a continuous fill; a centered day separator ('Today', 'Yesterday', date) inserts between days using divider.strong.

The 300 px cap is removed: the thread becomes the full-height center pane (presentation-only change to the container at inbox.rs:2306-2314). Role colors come exclusively from Table 9-1; an agent reply and an AI answer must differ at arm's length - fill, accent edge, and badge - eliminating the shared-tone defect of inbox.rs:2871/2875.

9.3 AI answer card

- Identity: sparkle glyph + 'AI' chip in accent.ai; never the Agent tone.
- Provenance line under the header: 'generated locally - model ', plus source count when the answer drew on indexed knowledge.
- Actions row: Insert into composer / Regenerate / Copy - the agent-agent pattern from current AI-UX libraries (provenance + user control + explicit hand-off).
- Applies to: thread AI entries, the Copilot suggested-reply card, and composer rewrite/translate results (which gain a persistent 'AI edited' chip with one-click revert instead of silent replacement).

9.4 Conversation list row

Intercom-list anatomy with the app's existing fields only: customer name (13 px semibold) as the primary line; one-line snippet (12 px muted) as secondary; relative timestamp right-aligned; status chip; assignee initials at the right edge (field already in ConversationListItem - core/inbox.rs:68-76); 56 px rows, virtualized via ScrollArea::show_rows; hover raises the row, selection uses bubble.agent tint. The toolbar becomes a segmented All / Active / Pending / Closed control; the pager is restyled but keeps its 50-per-page behavior (pagination mechanics are out of scope).

9.5 Composer

- Reply | Note tabs (Intercom convention: if the last thread entry is a note, Note opens by default) - note mode renders the editor on bubble.note tint so private intent is visible while typing.
- Saved Replies: an insert popover inside the composer (CRUD stays on Operations); inserting never clears the draft silently.
- Per-send status and assignee selectors beside the send button ('reply & close' without leaving the composer) - behavior-neutral: same job channel, richer one-shot payload.
- Drafts persist to SQLite through the existing reply:{remote}:draft: job infrastructure (core already has the channel; the UI currently never enqueues - inbox.rs:373-390).

9.6 Copilot panel

The single dense card (inbox.rs:1473-1589) splits into stacked, typed cards with distinct section headers: Signals (chips row), What the customer needs (summary bullets), Open questions, Similar past issues (each with number, subject, score chip, resolution line), and Suggested reply rendered as a proper AI answer card (9.3) with an Insert button - replacing the current nested mini-scroll of hint text. Visual weight descends from actionable to informational; the suggested reply is the only card with a primary action.

9.7 QA gates

- egui_kittest snapshots for message_card (one per role), list row states, composer tabs - run in CI next to the existing SPP_SMOKE boot-chain gate.
- Golden screenshot set regenerated through the existing SPP_SHOT_* harness (29 PNG baselines in qa-redesign/screenshots/ are the 'before').
- xtask color-lint: zero colors outside design.rs tokens (the 6 strays in settings.rs:1820-1826 are absorbed into tokens).
- AccessKit labels on every new interactive element (roles already set on nav and list rows are the pattern - app.rs:1046-1048, inbox.rs:1409-1418).
- The blind test: a person who has never seen the app must correctly segment a 20-message thread and identify which replies were AI's, first try.

10. The Interaction Track (Fast-Follow)

The reskin deliberately does not touch the data path; the freezes, hot loops and missing triage mechanics remain until the interaction track ships. It is summarized here (full treatment in the companion UX Recovery Plan work) so sequencing is explicit: reskin first - highest visible payoff, lowest risk, builds the component vocabulary the interaction track then consumes.

Step	Content	Depends on
U2-I1 Async data path	All inbox queries off the UI thread (worker + snapshot cache); hot loops get explicit reload triggers; sort index; FTS5 for inbox search	Reskin merged; rides the same migration train as the earlier hot-fix list
U2-I2 Triage	Unread state (schema addition), Mine / Unassigned views, multi-select + bulk assign/close, sort control	I1
U2-I3 Deep links	Palette carries the id through (fix palette.rs:92-96); recents	I1
U2-I4 Keyboard layer	j/k navigation, next-conversation, action keys, cheatsheet	I2

Table 10-1. Interaction track summary (post-reskin).

11. Metrics, Effort and Sequencing

The baseline is measured in R0 (five-task SUS plus a heuristic evaluation scored on the Nielsen Norman 10-point scale, using the existing 29-PNG screenshot set); the same instrument re-runs in R5. Targets:

Metric	Today (observed in code)	R5 target
Thread segmentation ('where does reply 2 end?')	Impossible by construction - no message boundaries exist	100% first-try on blind test
AI vs human reply, blind identification	Chance level - identical badge tone (inbox.rs:2871/2875)	100%
Time to find the customer's last message	Baseline set in R0	-70%
Colors outside tokens	6 strays (settings.rs:1820-1826)	0
Screenshot regressions vs golden set	n/a	0
SUS (same five tasks)	Baseline set in R0	At least 68 (50th percentile); stretch 75

Table 11-1. Success metrics.

Effort: one product designer (contract is sufficient) plus one Rust developer, about eight weeks elapsed part-time, with R0 starting immediately from existing screenshots. Artifacts live in the repository (/design spec, wireframes, token diff) so they are reviewable like code. Sequencing against the earlier roadmap is unchanged: the reskin consumes no Phase 0/1 work; the interaction track's async data path joins the existing hot-fix migration train once the reskin lands.

12. Do-NOT List

- **Do not touch schema, sync, or features.** 'Keep everything the same' is the contract; any behavioral change voids it and re-opens review.
- **Do not iMessage-ify the thread.** Alternating left/right bubbles are the consumer-messenger convention; agent workspaces use typed full-width blocks - and one-column egui renders those best.
- **Do not copy Intercom's palette blindly.** Adopt the semantic layer; keep zinc and the existing dark theme.
- **Do not hand-paint a single color outside tokens.** The color-lint gate exists to fail the build.
- **Do not bundle perf fixes into the reskin.** Separate track, or neither ships cleanly.
- **Do not redesign IA or consolidate pages in this workstream.** Out of scope by definition - the reskin must be reviewable as zero-behavior-change.
- **Do not skip the baseline.** Without R0 numbers, 'recovered' is unfalsifiable - the exact sin that let 0/10 persist unseen.

A. Appendix A. Receipt Index and Source Register

Repository at commit ee6b35a (branch main, kimpearce888/SupportOS-Plus). Paths relative to the workspace root; the audit clone is /home/z/supportos-plus-audit. Web sources were fetched and read on 28 October 2026.

A.1 Receipt index

Finding	Receipts (file:line)
Flat thread rendering (no message object)	crates/app-shell/src/pages/inbox.rs:2858-2930 (thread_entry_ui); divider at 2929; badge tones at 2869-2878
AI badge shares Agent tone	inbox.rs:2871 (Agent) vs 2875 (AI, BadgeTone::Info)
Quoted history inline as weak text	inbox.rs:2916-2918
300 px thread window; unlimited thread load	inbox.rs:2306-2314; crates/core/src/inbox.rs:326-341
Raw UTC compact timestamps	inbox.rs:2797-2801 (compact_time)
UI-thread SQL per interaction	inbox.rs:526-529, 551, 574, 624, 676; crates/core/src/db.rs:184 (with_read)
Per-frame hot loops (empty list; broken detail)	inbox.rs:1051-1056; 1060-1062
Non-virtualized list; 66 px hand-painted rows	inbox.rs:1288-1290, 1293
No unread / assignee filter hardcoded None	core/inbox.rs:59-81 (row struct); inbox.rs:121
Saved Replies unreachable from composer	operations.rs:746-747 vs zero hits for 'saved' in inbox.rs

Finding	Receipts (file:line)
Drafts session-only	inbox.rs:373-390 (StashedDraft; stash never outlives session)
Attachments downloaded but never shown	crates/core/src/workers.rs:3857-3918
Palette deep-links discard the id	palette.rs:92-96 (customers/knowledge: 98-106)
Pagination 50/page prev-next; fixed sort	inbox.rs:92, 1427-1435; core/inbox.rs:234; migrations.rs:100-105
LIKE-scan inbox search vs FTS5 engine	core/inbox.rs:197-201; crates/core/src/search.rs
Copilot single dense card	inbox.rs:1473-1589
Token system and components (kept)	design.rs:1-10, 104-179, 547-911; toasts.rs:39-46; pages/mod.rs:47-75
Magic numbers per call site	inbox.rs:1266 (360), 1293 (66), 2309 (300); app.rs:721 (216), 738 (52), 782 (22/14)
Fonts without CJK; ~460 hardcoded strings	design.rs:210-218; rg counts (button 102, hint 38, hover 29, RichText 21)
Screenshot harness and smoke gate	app.rs:1180-1189; qa-redesign/screenshots/ (29 PNGs); smoke.rs:33-42

Table A-1. Claim-to-receipt index (commit ee6b35a).

A.2 Source register

ID	Source (accessed 28 October 2026)	Used for
S1	Intercom Help - 'The Inbox explained' (intercom.com/help/en/articles/6258745)	Chapter 6 shared-workspace definition
S2	Fin Academy - 'Organizing your Intercom inbox with views' (academy.fin.ai)	Chapter 6 views concept
S3	Intercom Help - 'Loop teammates or teams into conversations' / 'Start a conversation from the Inbox'	Note conventions; composer defaults to Note
S4	everyonesdesign - Intercom Design System brand analysis (canvas #f5f1ec)	Chapter 6 brand-layer caveat
S5	WaysConf 2025 - Thom Rimmer, 'Designing Fin - the #1 AI Agent for Customer Service'	Chapter 6 AI identity
S6	Zendesk Support - 'About the Zendesk Agent Workspace'; internalnote.com AI-agent tickets analysis (Nov 2025)	Agent-workspace thread conventions
S7	setproduct - '16 AI agent UI design patterns' (Sep 2026); aiuxplayground pattern library; orangeloops '9 UX patterns for trustworthy AI assistants' (Jul 2025)	Section 9.3 AI card pattern
S8	Superhuman keyboard shortcuts docs + cmdk.email comparison (Aug 2026)	Chapter 6 keyboard-first bar
S9	Brooke, SUS - A quick and dirty usability scale (digital.ahrq.gov); Wikipedia SUS; PMC12066864 psychometric evaluation (2025); uxia.app guide (Mar 2026)	Chapter 6 / Table 11-1 SUS benchmarks (68 / 80.3 / 50)
S10	Nielsen Norman Group - 10 Usability Heuristics (nngroup.com/articles/ten-usability-heuristics)	R0/R5 heuristic evaluation instrument
S11	egui docs - ScrollArea::show_rows 10,000-row example (docs.rs/egui); egui_virtual_list crate	Section 9.4 virtualization

ID	Source (accessed 28 October 2026)	Used for
S12	egui_kittest crate (snapshot testing on AccessKit, egui 0.33+); egui 0.33 release notes	Section 9.7 QA gates
S13	Eleken - 'Legacy App Redesign: Modernize Without Breaking User Trust' (9-step framework)	Phased-method grounding
S14	smart-interface-design-patterns - 'Big-Bang Redesigns vs Legacy Modernization'	Do-NOT list item 6 rationale
S15	Chat-UI craft sources - multitaskai chat design trends (2025); CometChat message-bubble styling docs	Section 9.2 bubble craft

Table A-2. Source register.